

REST vs GraphQL

repo analysis & key points · [rest-vs-graphql](#) · Spring Boot 4.0.6 · Java 25 · WebFlux + Spring GraphQL · R2DBC MySQL

One service serves both the API and the demo page. The pitch is **not** “GraphQL beats REST” — it is: **what changes when clients choose their own response shape?**

01 · WHAT IT IS

Shape	REST GET + GraphQL POST against one MySQL bookstore
Backend	WebFlux (reactive Netty), Spring for GraphQL 2.0.3
Data	R2DBC MySQL via reactive <code>DatabaseClient</code>
Seed	200 authors · 10,000 books · 25,054 reviews

Seeding is idempotent: each boot verifies counts and rebuilds only when data is missing or stale. Three consecutive boots logged the same 25,054 reviews, proving the data is not regenerated on boot.

02 · ENDPOINTS

GET	/api/rest-vs-graphql/catalog/full light?limit=N
GET	/api/rest-vs-graphql/book/{id}
GET	/api/rest-vs-graphql/status
POST	/graphql
	<code>books(limit:Int=10) { books meta{dbCalls rowsLoaded elapsedMs} }</code>

Catalog and book replies carry `Cache-Control: max-age=60, public; status is no-store`. REST uses a `{status,data}` envelope.

03 · WHAT THE DEMO PROVES

- Over-fetching is visible:** full REST sends **11 fields**, the card uses **5 → 6 extra (55%)**.
- REST can be lean too:** light REST sends exactly the 5 fields the card needs, which blocks the lazy “REST is inherently wasteful” takeaway.
- Field evolution:** toggling `description` is a GraphQL query edit, but a backend contract change for light REST.
- Caching asymmetry:** GET inherits browser/CDN caching for free; GraphQL POST does not by default.

04 · KEY FILES

REST ctrl	<code>controller/RestVsGraphqlController.java</code>
REST svc	<code>service/RestVsGraphqlService.java</code>
GQL ctrl	<code>controller/BookGraphqlController.java</code>
GQL svc	<code>service/BookGraphqlService.java</code>
Schema	<code>resources/graphql/schema.graphqls</code>
Seed	<code>service/BookStoreDataInitializer.java</code>
Page	<code>numfeel-site/pages/rest-vs-graphql/</code>

05 · RECOMMENDATION

Keep REST for stable, cacheable public APIs. Use GraphQL behind a BFF where client flexibility and cross-resource composition justify the guardrails. Batch before launch; add depth and complexity limits, persisted queries and per-query metrics before exposing it.

06 · SERVER COST — THE PUNCHLINE

GraphQL selection	SQL calls
scalar fields only	1
+ author	1 + N
+ reviews	1 + N
+ author + reviews	1 + N + N

Measured at `limit=10` with both nested: **21 SQL calls**, 48 rows, 27–72 ms. At `limit=100` the same shape predicts **201** calls.

This N+1 model is deliberate for teaching, and it is also the honest operational risk: production GraphQL needs batching, depth and complexity limits, and persisted queries.

07 · MEASURED ON THIS BUILD

Request (limit=5)	wire bytes	fields
REST full	2,138	11
REST light	608	5
GraphQL	674	selected

Latency moves run to run (GraphQL 53/36/27 ms) and byte counts come from the live payload, so the page shows measurements rather than constants. Only the “theoretical prediction” is model-based, and the UI labels it as such.

Console cross-check: with dev logging on, the 21-call query prints exactly **21** Executing SQL statement lines, so the cost panel can be audited live.

08 · RUN & VERIFY

<pre>START_MYSQL=1 ./run-rest-vs-graphql-demo.sh open http://localhost:8080/pages/rest-vs-graphql/ curl -s localhost:8080/api/rest-vs-graphql/status -> {"dataReady":true,"authors":200,"books":10000,"reviews":25054}</pre>

The `dev` profile logs every request and every SQL statement; the default profile stays silent. Prod demo: `numfeel.996.ninja/pages/rest-vs-graphql/`. Add `?api=http://localhost:8080` to point a local page at a local API.

09 · WHAT WE’VE DONE

- Extracted the feature into its own repo; trimmed the pom, config and README and translated the last Chinese strings — the repo is all English now.
- Hardened the launcher: `NAME=VALUE` arguments are honoured (they were silently ignored, which caused the empty-password failure), bare words are rejected, and `-h` was added. Verified under bash and zsh.
- Added dev-only request and SQL logging so the UI cost numbers can be audited in the console.
- Verified end to end: 12 backend tests, 19 frontend tests (`engine.js`), live REST and GraphQL, cache headers, and the seeded dataset.

All figures were measured against the running build: byte counts are wire sizes, SQL counts come from the server’s own `meta` counters, seed counts from the status endpoint.